To fully understand a `CommittableSink`, you need to take a step back and understand `plainSink` a little better, among other things. This builds on a stage called `flexiFlow`, which, as the name suggests, is not a sink. Let's look at the implementation of `plainSink`:

```
package akka.kafka.scaladsl

object Producer {
  ...

  def plainSink[K, V](settings: ProducerSettings[K, V]):
        Sink[ProducerRecord[K, V], Future[Done]] =
    Flow[ProducerRecord[K, V]]
      .map(Message(_, NotUsed))           ❶
      .via(flexiFlow(settings))           ❷
      .toMat(Sink.ignore)(Keep.right)     ❸
  ...
}
```

❶ Wraps into ProducerMessage.Message

❷ Uses flexiFlow

❸ Ignores the result of the flexiFlow and keeps the materialization of the Sink.ignore. That is, Future[Done].

This is interesting for a couple of reasons. First, the implementation starts with a `map` that wraps each element that reaches the `plainSink` into a `ProducerMessage.Message`. The type of this message is a `ProducerMessage.Envelope`:

```
package akka.kafka

object ProducerMessage {
  ...

  final case class Message[K, V, +PassThrough](
      record: ProducerRecord[K, V],
      passThrough: PassThrough
  ) extends Envelope[K, V, PassThrough]
```